

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



**CHUYÊN ĐỀ NGHIÊN CỨU:
TRÍ TUỆ NHÂN TẠO (GK5937)**

Báo cáo tiểu luận

Đề tài: Các mô hình học sâu cho bài toán sinh ảnh

Sinh viên thực hiện: Trần Bảo Phúc - 2114452
Trần Minh Khoa - 2110278

HO CHI MINH CITY, DECEMBER 2024



Mục lục

1	TỔNG QUAN VỀ ĐỀ TÀI	3
1.1	Lý do chọn đề tài	3
1.2	Phạm vi nghiên cứu	3
1.2.1	Đối tượng nghiên cứu	3
1.2.2	Phạm vi nghiên cứu	3
1.3	Phương pháp nghiên cứu	4
1.4	Mục tiêu nghiên cứu	4
1.4.1	Mục tiêu tổng quát	4
1.4.2	Mục tiêu cụ thể	4
2	CƠ SỞ LÝ THUYẾT	5
2.1	Bài toán sinh ảnh	5
2.2	Mô hình sinh dữ liệu	7
2.2.1	Mô hình GAN	8
2.2.1.a	Nguyên lý hoạt động của GAN	9
2.2.1.b	Kiến Trúc của Mô Hình GAN	10
2.2.1.c	Quá trình huấn luyện	11
2.2.2	Mô hình Normalizing Flow	12
2.2.2.a	Ý tưởng chính của Normalizing Flow	12
2.2.2.b	Các biến đổi phổ biến trong Normalizing Flow	14
2.2.3	Mô hình VAE	14
2.2.3.a	Auto-Encoder (AE)	15
2.2.3.b	Variational Auto-Encoder	16
2.2.4	Mô hình Diffusion	17
2.2.4.a	Quá trình khuếch tán thuận	18
2.2.4.b	Quá trình khuếch tán nghịch	18
2.2.4.c	Khuếch tán hướng dẫn bằng mô hình phân loại (classifier guided diffusion) và không có mô hình phân loại (classifier free guidance)	19
2.3	Phương pháp đánh giá cho bài toán sinh ảnh	19
2.3.1	Frechet Inception Distance (FID)	20
3	HIỆN THỰC CÁC MÔ HÌNH VÀ ĐÁNH GIÁ KẾT QUẢ	21
3.1	Tập dữ liệu	21
3.2	Tổng quan hiện thực và kết quả của các mô hình	22
3.2.1	Mô hình GAN	22
3.2.1.a	Cấu hình	22
3.2.1.b	Kiến trúc	23



3.2.1.c	Kết quả	23
3.2.2	Mô hình Normalizing Flow với RealNVP	25
3.2.2.a	Cấu hình	25
3.2.2.b	Kiến trúc	26
3.2.2.c	Kết quả	26
3.2.3	Mô hình VAE	27
3.2.3.a	Cấu hình	27
3.2.3.b	Kiến trúc	28
3.2.3.c	Kết quả	28
3.2.4	Mô hình Diffusion	30
3.2.4.a	Cấu hình	30
3.2.4.b	Kiến trúc	31
3.2.4.c	Kết quả	32
4	SO SÁNH, ĐÁNH GIÁ KẾT QUẢ GIỮA CÁC MÔ HÌNH	33
5	TỔNG KẾT	35
	Tài liệu tham khảo	36

1 TỔNG QUAN VỀ ĐỀ TÀI

1.1 Lý do chọn đề tài

Bài toán sinh ảnh (Image Generation) không chỉ đóng vai trò quan trọng trong nghiên cứu trí tuệ nhân tạo, mà còn là nền tảng của nhiều ứng dụng thực tiễn, từ sáng tạo nội dung, phục hồi dữ liệu hình ảnh, cho đến hỗ trợ chẩn đoán y tế. Những tiến bộ từ các mô hình học sâu, bao gồm Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), và Diffusion Models, đã góp phần đáng kể vào việc tạo ra các hình ảnh có chất lượng cao và đa dạng.

Tuy nhiên, mỗi mô hình lại có cách tiếp cận riêng, với những ưu điểm và hạn chế khác nhau, ảnh hưởng đến hiệu quả ứng dụng trong từng ngữ cảnh. Ví dụ, GANs nổi bật nhờ khả năng tạo hình ảnh chi tiết, nhưng quá trình huấn luyện thường không ổn định. Trong khi đó, Diffusion Models vượt trội về độ chính xác nhưng đòi hỏi tài nguyên tính toán lớn, khiến chúng khó triển khai trong các hệ thống có giới hạn về tài nguyên.

Việc tổng hợp, phân tích và so sánh các mô hình này là cần thiết để làm rõ sự khác biệt về hiệu quả, độ phức tạp, và khả năng ứng dụng. Điều này không chỉ giúp hiểu sâu hơn về các mô hình hiện tại mà còn mở ra hướng cải tiến trong tương lai, đáp ứng tốt hơn các yêu cầu thực tế. Vì lý do đó, nhóm quyết định thực hiện đề tài này nhằm xây dựng một cái nhìn toàn diện và hệ thống về các mô hình sinh ảnh.

1.2 Phạm vi nghiên cứu

1.2.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu của tiểu luận bao gồm:

- Đề tài tập trung vào 4 mô hình sinh ảnh phổ biến, bao gồm: Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), Diffusion Models, Normalizing Flows.
- Nhóm so sánh hiệu quả 4 mô hình dựa trên phương pháp đánh giá phổ biến là Frechet Inception Distance (FID).

1.2.2 Phạm vi nghiên cứu

Về phạm vi nghiên cứu, nhóm sẽ tìm hiểu và hiện thực các mô hình sinh dữ liệu cơ bản cho bài toán sinh ảnh với dữ liệu MNIST.

1.3 Phương pháp nghiên cứu

Phương pháp nghiên cứu được sử dụng trong đề tài được thực hiện qua các bước sau:

- Lên ý tưởng thực hiện đề tài
- Tham khảo các tài liệu liên quan
- Tái hiện thực thuật toán được sử dụng trong tài liệu tham khảo
- Đánh giá, so sánh và nhận xét kết quả đạt được

1.4 Mục tiêu nghiên cứu

1.4.1 Mục tiêu tổng quát

Đề tài nhằm mục đích tổng hợp, phân tích và so sánh các mô hình sinh ảnh phổ biến trong học sâu, từ đó đánh giá hiệu quả và khả năng ứng dụng của từng mô hình trong các ngữ cảnh khác nhau.

1.4.2 Mục tiêu cụ thể

- Tổng hợp kiến thức: Nghiên cứu lý thuyết và các cơ chế hoạt động chính của các mô hình sinh ảnh bao gồm GANs, VAEs, Diffusion Models và Normalizing Flow.
- Thực nghiệm:
 - Cài đặt và huấn luyện một số mô hình sinh ảnh trên các tập dữ liệu MNIST.
 - So sánh chất lượng ảnh được sinh ra bởi các mô hình dựa trên các tiêu chí về chất lượng và độ đa dạng.
- Đánh giá và phân tích:
 - Đưa ra nhận xét chi tiết về ưu điểm và hạn chế của từng mô hình.
 - Đề xuất hướng cải tiến hoặc các ứng dụng tiềm năng cho các mô hình sinh ảnh.

2 CƠ SỞ LÝ THUYẾT

2.1 Bài toán sinh ảnh

Bài toán sinh ảnh (Image Generation) là một lĩnh vực nghiên cứu quan trọng trong lĩnh vực trí tuệ nhân tạo. Mục tiêu là xây dựng các mô hình có khả năng học được phân phối xác suất tiềm ẩn từ tập dữ liệu hình ảnh ban đầu, từ đó tạo ra các hình ảnh mới, có độ chân thực cao và thỏa mãn các đặc trưng của dữ liệu gốc. Điều này đòi hỏi mô hình phải học được được cả các đặc trưng bề mặt như màu sắc, kết cấu, lẫn các đặc trưng sâu hơn như cấu trúc ngữ nghĩa (semantic structures) hoặc các mối quan hệ giữa các đối tượng trong hình ảnh.



Hình 1: Các ảnh được tạo ra bằng mô hình ControlNet dựa trên ảnh gốc đầu vào

Bài toán sinh ảnh có thể được phân loại thành:

- Unconditional Image Generation: Sinh hình ảnh mà không có bất kỳ điều kiện đầu vào nào, thường được sử dụng để mô phỏng phân phối tổng quát của tập dữ liệu.
- Conditional Image Generation: Sinh hình ảnh dựa trên các điều kiện đầu vào, chẳng hạn như sinh ảnh từ văn bản (Text-to-Image) hoặc chuyển đổi giữa các domain hình ảnh (Image-to-Image Translation).

Các mô hình sinh ảnh phổ biến hiện nay bao gồm Generative Adversarial Networks (GANs), nổi bật với các biến thể như StyleGAN và BigGAN, chuyên tạo ra hình ảnh chất lượng cao và chi tiết; Variational Autoencoders (VAEs), tập trung vào học phân phối tiềm ẩn và có khả năng sinh ảnh mượt mà nhưng kém chân thực hơn GANs; Diffusion Models, như DALL-E 2 và Stable Diffusion, sử dụng kỹ

thuật khuếch tán ngược để sinh ảnh rõ nét và đa dạng; và các mô hình dựa trên Transformers, điển hình là Imagen và Muse, kết hợp khả năng xử lý chuỗi dữ liệu để sinh ảnh từ văn bản hoặc các điều kiện khác. Những mô hình này đã đạt được nhiều tiến bộ vượt bậc, mở rộng khả năng ứng dụng AI trong nhiều lĩnh vực.

Các ứng dụng của bài toán sinh ảnh bao gồm:

- **Sáng tạo nội dung:** Các hệ thống như DALL-E hay Stable Diffusion có khả năng tạo ra tác phẩm nghệ thuật độc đáo hoặc mẫu thiết kế sáng tạo, phục vụ ngành công nghiệp giải trí và quảng cáo.
- **Tăng cường dữ liệu (Data Augmentation):** Bài toán sinh ảnh cung cấp một giải pháp hiệu quả để mở rộng tập dữ liệu trong các lĩnh vực như nhận dạng khuôn mặt, phân loại hình ảnh, hoặc phát hiện đối tượng.
- **Chỉnh sửa và phục hồi hình ảnh:** Các kỹ thuật như inpainting (phục hồi vùng ảnh bị che khuất) hay super-resolution (nâng cao độ phân giải) là những ví dụ điển hình trong việc cải thiện chất lượng hình ảnh.
- **Ứng dụng y tế:** Trong y học, các mô hình sinh ảnh giúp tái tạo hình ảnh từ các dữ liệu nhiễu, tạo ảnh giả lập để thử nghiệm, hoặc hỗ trợ phân tích chuyên sâu trong các bài toán chẩn đoán bằng hình ảnh.

Bài toán sinh ảnh đối mặt với nhiều thách thức, đòi hỏi sự phát triển không ngừng của các thuật toán và kiến trúc mạng. Các thách thức bao gồm:

- **Học phân phối phức tạp:** Không gian hình ảnh có số chiều lớn, với các đặc trưng phi tuyến tính phức tạp, khiến việc học và tái tạo phân phối xác suất của tập dữ liệu trở nên khó khăn.
- **Ổn định trong quá trình huấn luyện:** Các mô hình như GAN thường gặp vấn đề mất cân bằng giữa generator và discriminator, dẫn đến kết quả không ổn định hoặc không đạt chất lượng mong muốn.
- **Độ phức tạp tính toán:** Sinh ảnh với độ phân giải cao đòi hỏi tài nguyên tính toán lớn, đặc biệt với các mô hình hiện đại như Diffusion Models hoặc Transformer-based architectures.
- **Khả năng tổng quát hóa:** Mô hình cần đảm bảo rằng các hình ảnh sinh ra không chỉ phù hợp với tập dữ liệu huấn luyện mà còn có khả năng mở rộng sang các tập dữ liệu chưa từng thấy trước đó.

2.2 Mô hình sinh dữ liệu

Mô hình sinh dữ liệu là các mô hình học phân phối xác suất của dữ liệu, từ đó cho phép sinh ra các mẫu dữ liệu mới tương tự với phân phối thực tế. Trong lý thuyết xác suất, mô hình sinh thường tập trung vào việc học phân phối đồng thời $p(x, y)$, trong đó x là dữ liệu quan sát và y là thông tin bổ sung, ví dụ như nhãn hoặc điều kiện đi kèm. Tuy nhiên, trong nhiều trường hợp, không có y , mô hình sẽ chỉ học phân phối biên $p(x)$.

Mô hình sinh khi không có y

Khi không có thông tin y , mục tiêu của mô hình sinh là ước lượng và học phân phối $p(x)$, tức là phân phối của dữ liệu quan sát x . Cụ thể, ta muốn:

$$p_{\theta}(x) \approx p_{\text{data}}(x)$$

Trong đó:

- $p_{\text{data}}(x)$: Phân phối thực tế của dữ liệu.
- $p_{\theta}(x)$: Phân phối được học bởi mô hình, với tham số θ .

Hầu hết các mô hình sinh hiện đại đều dựa trên ý tưởng sử dụng biến ẩn z , với giả định rằng dữ liệu x được sinh ra từ một biến tiềm ẩn z . Khi đó, phân phối $p(x)$ có thể được mô tả như:

$$p(x) = \int p(x|z)p(z)dz$$

Trong đó:

- $p(z)$: Phân phối của biến tiềm ẩn z , thường là phân phối Gaussian chuẩn $\mathcal{N}(0, I)$.
- $p(x|z)$: Phân phối có điều kiện, được xác định bởi một mô hình (ví dụ: mạng nơ-ron).

Ngoài ra, một số phương pháp khác như Normalizing Flow không sử dụng biến ẩn z , mà thay vào đó ánh xạ trực tiếp dữ liệu x từ một không gian đơn giản hơn thông qua một ánh xạ khả nghịch.

Mô hình sinh khi có y

Khi có thông tin bổ sung y , mô hình sinh sẽ học phân phối có điều kiện $p(x|y)$, trong đó y đóng vai trò điều kiện để kiểm soát việc sinh dữ liệu x . Một số ví dụ phổ biến bao gồm:

- Sinh ảnh từ mô tả văn bản: y là văn bản mô tả, và mô hình học $p(x|y)$ để sinh ảnh phù hợp với nội dung văn bản.
- Tô màu ảnh: y là ảnh đen trắng, và mô hình học $p(x|y)$ để tô màu x tương ứng.

Ứng dụng của mô hình sinh

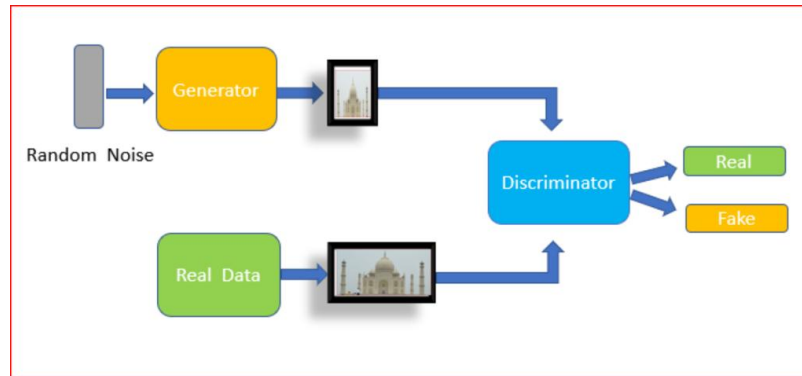
Các mô hình sinh học phân phối $p(x)$ hoặc $p(x|y)$ được sử dụng trong nhiều ứng dụng thực tiễn, bao gồm:

- **Sinh dữ liệu không giám sát:** Mô hình học $p(x)$ để sinh ra các mẫu dữ liệu mới tương tự với dữ liệu gốc mà không cần nhãn.
- **Khôi phục dữ liệu:** Các mô hình như Diffusion Models có thể học $p(x|y)$ để khôi phục dữ liệu bị mất (inpainting) hoặc cải thiện chất lượng ảnh.
- **Ứng dụng sáng tạo:** GAN và các mô hình tương tự được sử dụng để tạo ra nội dung sáng tạo như hình ảnh, âm nhạc, hoặc video.

2.2.1 Mô hình GAN

Generative Adversarial Networks (GAN) là một loại mạng học sâu được đề xuất bởi Ian Goodfellow và các cộng sự vào năm 2014, với mục tiêu sinh ra dữ liệu mới có đặc điểm tương đồng với dữ liệu thật. GAN hoạt động dựa trên sự cạnh tranh giữa hai mô hình: Generator (G) và Discriminator (D).

2.2.1.a Nguyên lý hoạt động của GAN



Hình 2: Sơ đồ hoạt động mô hình GAN

Mô hình GAN được xây dựng dựa trên ý tưởng của một trò chơi đối kháng (zero-sum game). Generator cố gắng tạo ra các mẫu dữ liệu giả sao cho Discriminator không thể phân biệt được chúng với dữ liệu thật. Trong khi đó, Discriminator học cách phân biệt giữa dữ liệu thật và dữ liệu giả do Generator tạo ra.

Hàm loss của GAN được biểu diễn như sau:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log (1 - D(G(z)))]$$

Trong đó:

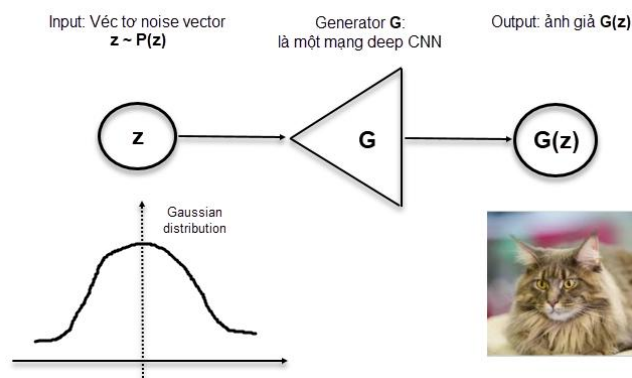
- $p_{\text{data}}(x)$ là phân phối của dữ liệu thật, và $p_g(x)$ là phân phối của dữ liệu giả do Generator sinh ra.
- $D(x)$ biểu diễn xác suất mà Discriminator đánh giá mẫu x là dữ liệu thật.
- $G(z)$ là dữ liệu giả được sinh ra từ một vector nhiễu $z \sim p_z(z)$, với $p_z(z)$ thường là phân phối chuẩn $\mathcal{N}(0, 1)$.

Quá trình huấn luyện GAN hướng đến việc làm cho $p_g(x)$ tiến gần đến $p_{\text{data}}(x)$. Khi đạt trạng thái cân bằng Nash, Discriminator không thể phân biệt được dữ liệu thật và dữ liệu giả, tức là $D(x) \approx 0.5$ cho mọi x .

2.2.1.b Kiến Trúc của Mô Hình GAN

Mô hình GAN (Generative Adversarial Network) bao gồm hai thành phần chính: Generator và Discriminator. Hai thành phần này hoạt động theo một cơ chế đối kháng, tức là Generator cố gắng tạo ra các mẫu dữ liệu giả sao cho Discriminator không thể phân biệt được chúng với dữ liệu thật, trong khi đó, Discriminator học cách phân biệt dữ liệu thật và dữ liệu giả.

Generator

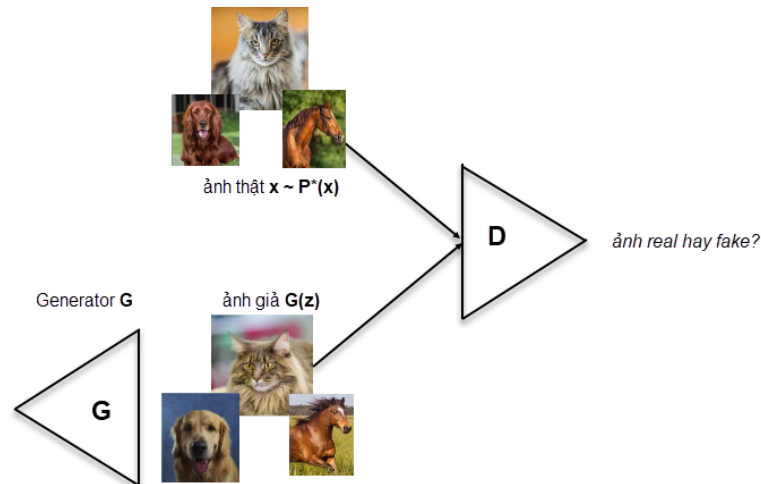


Hình 3: Sơ đồ kiến trúc của Generator

Generator là một mạng nơ-ron học cách tạo ra dữ liệu giả từ một vector nhiễu ngẫu nhiên z . Vector z thường được lấy từ một phân phối xác suất chuẩn. Generator có vai trò như một hàm ánh xạ từ không gian nhiễu sang không gian dữ liệu.

Kiến trúc của Generator thường bao gồm các tầng deconvolution hoặc các tầng fully connected, nhằm mở rộng dữ liệu từ vector nhiễu kích thước nhỏ thành dữ liệu có kích thước lớn như hình ảnh.

Discriminator



Hình 4: Sơ đồ kiến trúc của Discriminator

Discriminator là một mạng phân loại nhị phân, học cách phân biệt giữa dữ liệu thật và dữ liệu giả. Đầu vào của Discriminator có thể là một mẫu dữ liệu thật từ tập dữ liệu hoặc một mẫu giả do Generator tạo ra.

Kiến trúc của Discriminator thường sử dụng các tầng tích chập (convolutional layers) và các hàm kích hoạt như Leaky ReLU, với hàm sigmoid ở đầu ra để trả về xác suất mà mẫu đầu vào là thật.

2.2.1.c Quá trình huấn luyện

Quá trình huấn luyện GAN bao gồm hai bước luân phiên:

Huấn Luyện Discriminator (D)

Tham số θ_D của Discriminator được cập nhật bằng cách tối ưu hóa hàm mất mát sau:

$$\frac{1}{m} \sum_{i=1}^m [\log D(x^{(i)}) + \log (1 - D(G(z^{(i)})))]$$

Trong đó:

- $x^{(i)}$ là mẫu dữ liệu thật,
- $G(z^{(i)})$ là mẫu dữ liệu giả được sinh ra từ Generator với đầu vào là vector nhiễu $z^{(i)}$,

- m là số lượng mẫu trong mỗi batch.

Huấn Luyện Generator (G)

Tham số θ_G của Generator được cập nhật bằng cách tối ưu hóa hàm mất mát sau:

$$\frac{1}{m} \sum_{i=1}^m \log D(G(z^{(i)}))$$

Trong đó:

- $G(z^{(i)})$ là dữ liệu giả được sinh ra từ vector nhiễu $z^{(i)}$,
- m là số lượng mẫu trong mỗi batch.

Quá trình này được lặp lại qua nhiều epoch, giúp G tạo dữ liệu ngày càng giống dữ liệu thật và D phân biệt tốt hơn giữa thật và giả, đến khi mô hình hội tụ.

Việc huấn luyện GAN gặp nhiều khó khăn, điển hình là hiện tượng "mode collapse", khi Generator chỉ sinh ra một vài loại dữ liệu và không đảm bảo được tính đa dạng. Ngoài ra, việc hội tụ trong huấn luyện GAN thường không ổn định do tính chất đối kháng của hai mạng.

2.2.2 Mô hình Normalizing Flow

Normalizing Flow (NF) là một phương pháp mạnh mẽ trong lĩnh vực học sâu để biểu diễn các phân phối xác suất phức tạp thông qua chuỗi các biến đổi ánh xạ một phân phối đơn giản (thường là Gaussian chuẩn) sang phân phối mục tiêu. Mô hình NF đặc biệt hữu ích trong các bài toán sinh dữ liệu và tối ưu hóa mật độ xác suất.

2.2.2.a Ý tưởng chính của Normalizing Flow

Normalizing Flow tận dụng chuỗi các biến đổi khả nghịch để chuyển đổi từ một phân phối đơn giản $p(z)$ sang một phân phối phức tạp hơn $p(x)$. Các thành phần chính của NF bao gồm:

- **Biến đổi khả nghịch (Invertible Transformations):**
 - Mỗi phép biến đổi được biểu diễn bởi một hàm f , đảm bảo khả năng ánh xạ và nghịch đảo.
 - Nếu $z = f(x)$, thì x có thể được phục hồi bằng $x = f^{-1}(z)$.

- **Thay đổi mật độ xác suất (Change of Variables):** Với một chuỗi biến đổi $x \rightarrow z_1 \rightarrow z_2 \rightarrow \dots \rightarrow z_k$, mật độ xác suất $p(x)$ có thể được tính thông qua định lý biến đổi:

$$p(x) = p(z_k) \left| \det \left(\frac{\partial z_k}{\partial x} \right) \right|$$

Trong đó:

- $p(z_k)$: Phân phối cơ sở (base distribution), thường là Gaussian chuẩn.
- $\det \left(\frac{\partial z_k}{\partial x} \right)$: Định thức Jacobian của biến đổi, biểu diễn độ co giãn của không gian.

Kiến trúc của Normalizing Flow

Mô hình Normalizing Flow được xây dựng từ chuỗi các hàm biến đổi khả nghịch $f_1, f_2, \dots, f_k$. Các bước chính bao gồm:

1. **Chọn phân phối cơ sở $p(z)$:** Đây là một phân phối đơn giản như Gaussian chuẩn $\mathcal{N}(0, I)$.
2. **Ánh xạ dữ liệu x sang không gian latent z :**

$$z_k = f_k \circ f_{k-1} \circ \dots \circ f_1(x)$$

3. **Tính mật độ xác suất của x :**

$$\log p(x) = \log p(z_k) - \sum_{i=1}^k \log \left| \det \left(\frac{\partial f_i}{\partial z_{i-1}} \right) \right|$$

Trong đó, $z_0 = x$ và z_k là đầu ra cuối cùng sau chuỗi biến đổi.

Hàm mất mát

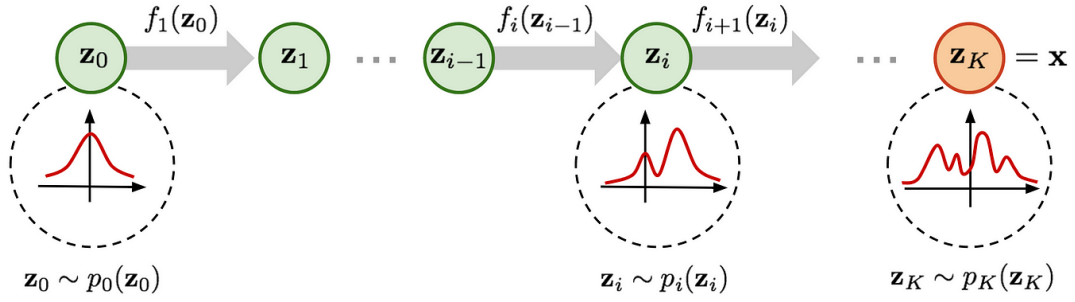
Hàm mất mát của NF tối ưu hóa *log-likelihood* của dữ liệu đầu vào x :

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \left[\log p(z_k) - \sum_{j=1}^k \log \left| \det \left(\frac{\partial f_j}{\partial z_{j-1}} \right) \right| \right]$$

Trong đó:

- $\log p(z_k)$: Mật độ xác suất của phân phối cơ sở.

- $\log \left| \det \left(\frac{\partial f_j}{\partial z_{j-1}} \right) \right|$: Định thức Jacobian của từng phép biến đổi.



Hình 5: Mô tả về kiến trúc Normalizing Flow

2.2.2.b Các biến đổi phổ biến trong Normalizing Flow

Một số phép biến đổi phổ biến trong NF bao gồm:

- **Affine Coupling Layer:**

- Phân chia x thành hai phần x_1 và x_2 .
- Ánh xạ x_2 qua một phép biến đổi phi tuyến s và t :

$$z_1 = x_1, \quad z_2 = x_2 \cdot \exp(s(x_1)) + t(x_1)$$

- Định thức Jacobian dễ tính toán: $\det(J) = \prod \exp(s(x_1))$.

- **Invertible 1x1 Convolution:** Biến đổi ma trận qua phép nhân tích chập, đảm bảo khả nghịch nhờ sử dụng ma trận có định thức khác 0.

- **Planar và Radial Flows:** Sử dụng các hàm phi tuyến đơn giản để ánh xạ không gian latent.

2.2.3 Mô hình VAE

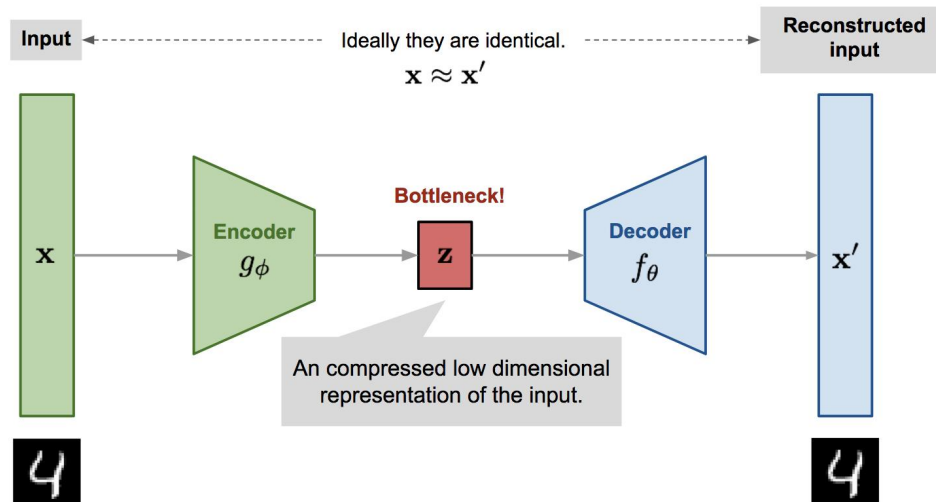
Variational Auto-Encoder (VAE) là một biến thể của mô hình Auto-Encoder mà ở đây các giả định về xác suất được thêm vào để mô hình có khả năng sinh ra dữ liệu mới.

2.2.3.a Auto-Encoder (AE)

Auto-Encoder (AE) là một loại mạng nơ-ron sử dụng trong học không giám sát với mục tiêu chính là tái tạo lại dữ liệu đầu vào từ một biểu diễn nén (*latent representation*). AE bao gồm hai thành phần chính:

- **Bộ mã hóa (Encoder):** Chuyển đổi dữ liệu đầu vào x thành một véc-tơ ẩn z trong không gian latent.
 - Quá trình này thường được biểu diễn bằng hàm $z = f(x)$, trong đó f là một mạng nơ-ron hoặc một hàm phi tuyến.
 - z là biểu diễn nén của dữ liệu, có số chiều thấp hơn nhiều so với dữ liệu gốc.
- **Bộ giải mã (Decoder):** Tái tạo lại dữ liệu x' từ véc-tơ ẩn z .
 - Hàm giải mã thường được biểu diễn bằng $x' = g(z)$, với g là một mạng nơ-ron.
 - Mục tiêu của mô hình là giảm thiểu sai số tái tạo $\|x - x'\|$.

Nhược điểm lớn của Auto-Encoder truyền thống là véc-tơ ẩn z chỉ là một giá trị cố định, không biểu thị được tính ngẫu nhiên hoặc sự phân phối của dữ liệu gốc. Điều này khiến AE không phù hợp để sinh dữ liệu mới.



Hình 6: Mô tả về kiến trúc Auto-Encoder

2.2.3.b Variational Auto-Encoder

Variational Auto-Encoder (VAE) là một biến thể của Auto-Encoder, được phát triển để khắc phục nhược điểm trên và mở rộng khả năng của AE sang lĩnh vực mô hình sinh. Điểm khác biệt chính của VAE là:

- **Mã hóa thành phân phối xác suất:** Thay vì ánh xạ dữ liệu x thành một véc-tơ ẩn cố định z , VAE ánh xạ x thành một phân phối xác suất $q(z|x)$, thường được giả định là phân phối chuẩn với trung bình μ và phương sai σ^2 . Bộ mã hóa trong VAE không phải học các giá trị z , mà lại tập học hai tham số μ và σ đặc trưng cho phân phối Gaussian ẩn.
- **Lấy mẫu ngẫu nhiên từ phân phối:** Sau khi có được μ và σ , véc-tơ ẩn z được lấy mẫu ngẫu nhiên từ phân phối $\mathcal{N}(\mu, \sigma^2)$. Quá trình này thêm tính ngẫu nhiên, giúp mô hình có khả năng sinh ra dữ liệu đa dạng.
- **Tái tạo và sinh dữ liệu:** Bộ giải mã sử dụng z để tái tạo lại x' , giống như trong AE. Tuy nhiên, nhờ việc lấy mẫu từ phân phối, VAE có thể sinh ra những dữ liệu x' hoàn toàn mới thay vì chỉ tái tạo lại dữ liệu đầu vào.

Hàm mất mát

Hàm mất mát trong VAE được thiết kế để cân bằng giữa:

- **Sự tái tạo (Reconstruction Loss):** Đảm bảo dữ liệu x' tái tạo giống với dữ liệu x , thường được đo bằng phương pháp như MSE (Mean Squared Error) hoặc *log-likelihood*.
- **Sự tương thích so với phân phối tiên nghiệm (Prior Matching Regularization):** Đảm bảo phân phối $q(z|x)$ (phân phối do mô hình học được) gần với phân phối chuẩn $p(z) = \mathcal{N}(0, I)$.

Dưới đây ta có công thức chung của hàm mất mát tổng quát:

$$\mathcal{L} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x)||p(z))$$

Trong đó:

- $\mathbb{E}_{q(z|x)}[\log p(x|z)]$: Sai số tái tạo, biểu thị khả năng của mô hình tái tạo lại dữ liệu đầu vào x từ véc-tơ ẩn z .
- $D_{KL}(q(z|x)||p(z))$: Khoảng cách KL giữa hai phân phối $q(z|x)$ và $p(z)$.

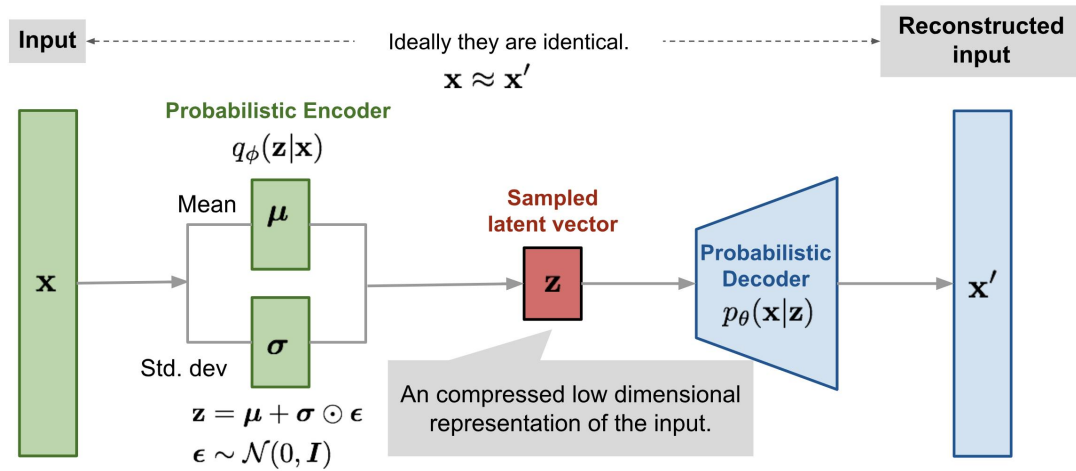
Nếu ta giả định $p(x|z)$ là một phân phối Gaussian với phương sai cố định σ^2 , thì hàm mất mát có thể được viết lại như sau:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \left[\frac{\|x_i - g(z_i)\|^2}{2\sigma^2} + D_{KL}(q(z|x_i)||p(z)) \right]$$

Trong đó:

- $g(z)$: Hàm giải mã (decoder), ánh xạ từ không gian latent z về không gian dữ liệu x .
- $\|x_i - g(z_i)\|^2$: Khoảng cách bình phương giữa dữ liệu gốc x_i và dữ liệu tái tạo $g(z_i)$, tương ứng với thành phần lỗi tái tạo.
- σ^2 : Phương sai cố định của phân phối $p(x|z)$, thường được coi là một siêu tham số.
- N : Số lượng mẫu dữ liệu trong một batch.

Khi tối ưu hóa hàm mất mát này, ta có thể học được cả các tham số của bộ mã hóa (*encoder*) và giải mã (*decoder*).



Hình 7: Mô tả về Variational Auto-Encoder

2.2.4 Mô hình Diffusion

Mô hình khuếch tán (Diffusion Model) sinh dữ liệu (thường được dùng cho dữ liệu ảnh) thông qua quá trình học nhiễu. Nó bao gồm hai quá trình là khuếch

tán thuận (forward diffusion) và khuếch tán nghịch (reverse diffusion). Quá trình khuếch tán thuận sẽ dần thêm nhiễu vào dữ liệu gốc theo thời gian, mục tiêu là biến đổi dữ liệu gốc ở phân bố phức tạp thành dữ liệu với phân bố đơn giản để thuận tiện cho việc lấy mẫu, thường là Gaussian $\mathcal{N}(0, I)$. Và như vậy, quá trình khuếch tán nghịch sẽ tìm cách khử nhiễu để sinh ra dữ liệu ở phân bố gốc.

2.2.4.a Quá trình khuếch tán thuận

- Với dữ liệu đầu vào là một hình ảnh, mô hình lần lượt thêm một lượng nhiễu nhỏ vào ảnh qua từng bước. Đầu ra $\mathbf{x}_{t+1}$ ở mỗi bước là một phân phối Gaussian với giá trị trung bình và phương sai phụ thuộc vào đầu vào $\mathbf{x}_t$ và giá trị $\beta_t \in (0, 1)$ được định nghĩa trước thông qua một lịch trình phương sai (variance schedule).

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}) \quad q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})$$

- Qua từng bước thêm nhiễu, dữ liệu dữ nguyên kích thước ban đầu. Cuối quá trình là một phân phối Gaussian đẳng hướng, tức là một hình ảnh hoàn toàn nhiễu cùng kích thước với hình ảnh đầu vào.

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$$

Trong đó, $\alpha_t = 1 - \beta_t$ và $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$

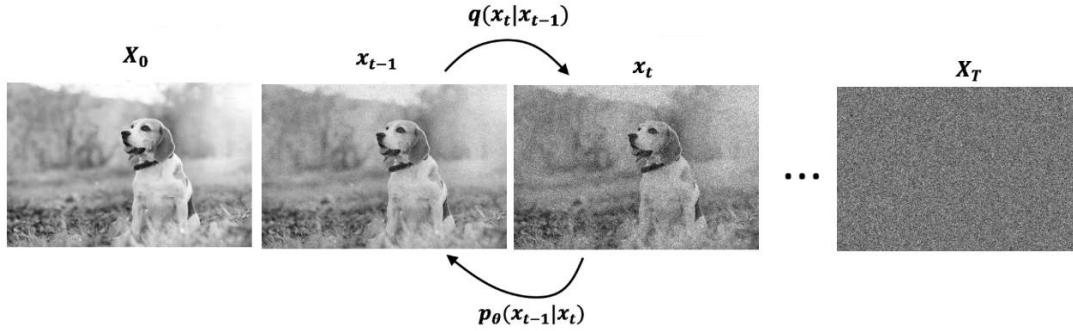
2.2.4.b Quá trình khuếch tán nghịch

- Đầu vào của quá trình này là một ảnh hoàn toàn nhiễu $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$. Mô hình lần lượt loại bỏ nhiễu khỏi hình qua từng bước để khôi phục lại hình ảnh ban đầu

$$p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) \quad p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

- Mô hình sẽ học nhiễu của quá trình thuận để khử nhiễu trong quá trình nghịch.

$$L_t^{\text{simple}} = \mathbb{E}_{t \sim [1, T], \mathbf{x}_0, \epsilon_t} \left[\|\epsilon_t - \epsilon_\theta(\mathbf{x}_t, t)\|^2 \right]$$



Hình 8: Quá trình khuếch tán

2.2.4.c Khuếch tán hướng dẫn bằng mô hình phân loại (classifier guided diffusion) và không có mô hình phân loại (classifier free guidance)

Trong quá trình khuếch tán ngược, thông thường ta muốn sinh ảnh dựa trên một điều kiện đầu vào y nào đó (mô tả văn bản, hình ảnh, nhãn, ...). Vì vậy ta cần điều chỉnh nhiễu học được qua từng bước khuếch tán để hướng dữ liệu đến gần điều kiện. Cuối quá trình khử nhiễu là hình ảnh mong muốn theo điều kiện y . Để làm được điều này, có hai cách giải quyết được đưa ra.

1. Thứ nhất là học một bộ phân loại (classifier) có khả năng phân loại để đưa ra nhãn của dữ liệu gốc. Ta sử dụng kết quả của mô hình phân loại này để điều chỉnh ϵ . Với cách này, ta không cần thiết phải huấn luyện lại mô hình khuếch tán nếu sẵn có trên tập dữ liệu. Tuy nhiên ta phải huấn luyện thêm một mô hình phân loại riêng biệt.

$$\bar{\epsilon}_{\theta}(\mathbf{x}_t, t) = \epsilon_{\theta}(\mathbf{x}_t, t) - \sqrt{1 - \bar{\alpha}_t} w \nabla_{\mathbf{x}_t} \log f_{\phi}(y|\mathbf{x}_t)$$

Trong đó, w là trọng số để kiểm soát mức độ ảnh hưởng của bộ phân loại.

2. Thứ hai là huấn luyện lại mô hình khuếch tán sao cho nó nhận đầu vào vừa là ảnh x_t , vừa là dữ liệu điều kiện y (ảnh, văn bản, nhãn).

$$\bar{\epsilon}_{\theta}(\mathbf{x}_t, t, y) = (w + 1)\epsilon_{\theta}(\mathbf{x}_t, t, y) - w\epsilon_{\theta}(\mathbf{x}_t, t)$$

2.3 Phương pháp đánh giá cho bài toán sinh ảnh

Trong bài toán Image Generation, việc đánh giá ảnh sinh ra thường xoay quanh hai yếu tố chính:

- Chất lượng ảnh: Ảnh được sinh ra phải có chất lượng cao, chân thực và phù hợp với phân phối của dữ liệu gốc.

- Độ đa dạng ảnh: Ảnh được sinh ra phải đại diện cho nhiều mẫu khác nhau trong dữ liệu, thay vì bị lặp lại hoặc tập trung vào một vài mẫu nhất định.

2.3.1 Frechet Inception Distance (FID)

Để tính FID, chúng ta sử dụng Inception network, nhưng chỉ giữ lại phần AvgPool và loại bỏ các lớp sau đó. Khi đó, đầu ra của mạng sẽ là một vector có kích thước 2048×1 . Với mỗi ảnh trong tập dữ liệu, ta sẽ nhận được một vector như vậy. Giả sử rằng tập dữ liệu có n ảnh, ta có thể thực hiện các bước sau:

Các bước tính FID:

1. Chạy ảnh trong tập dữ liệu qua Inception network:
 - Đối với n ảnh trong tập dữ liệu gốc, ta sẽ thu được n vector kích thước 2048×1 .
 - Dựa trên các vector này, ta xây dựng được một phân phối Gaussian đa chiều (multivariate Gaussian distribution) cho tập dữ liệu.
2. Chạy ảnh sinh ra qua Inception network:
 - Đối với n ảnh sinh ra từ mô hình, ta cũng thu được n vector kích thước 2048×1 .
 - Tương tự như với ảnh thực tế, ta xây dựng một phân phối Gaussian đa chiều cho các ảnh sinh ra.
3. So sánh phân phối của ảnh sinh ra và ảnh thực tế:
 - Để các ảnh sinh ra có sự phân phối giống với ảnh thực tế, ta mong muốn hai phân phối Gaussian này có các tham số mean và variance gần nhau.

FID được định nghĩa qua công thức sau:

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{Tr} \left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2} \right)$$

Trong đó:

- μ_r và μ_g lần lượt là mean vector của phân phối Gaussian ảnh thực tế và ảnh sinh ra.
- Σ_r và Σ_g là covariance matrices của phân phối Gaussian ảnh thực tế và ảnh sinh ra.
- Tr là phép toán trace, tính tổng các phần tử trên đường chéo của ma trận.

3 HIỆN THỰC CÁC MÔ HÌNH VÀ ĐÁNH GIÁ KẾT QUẢ

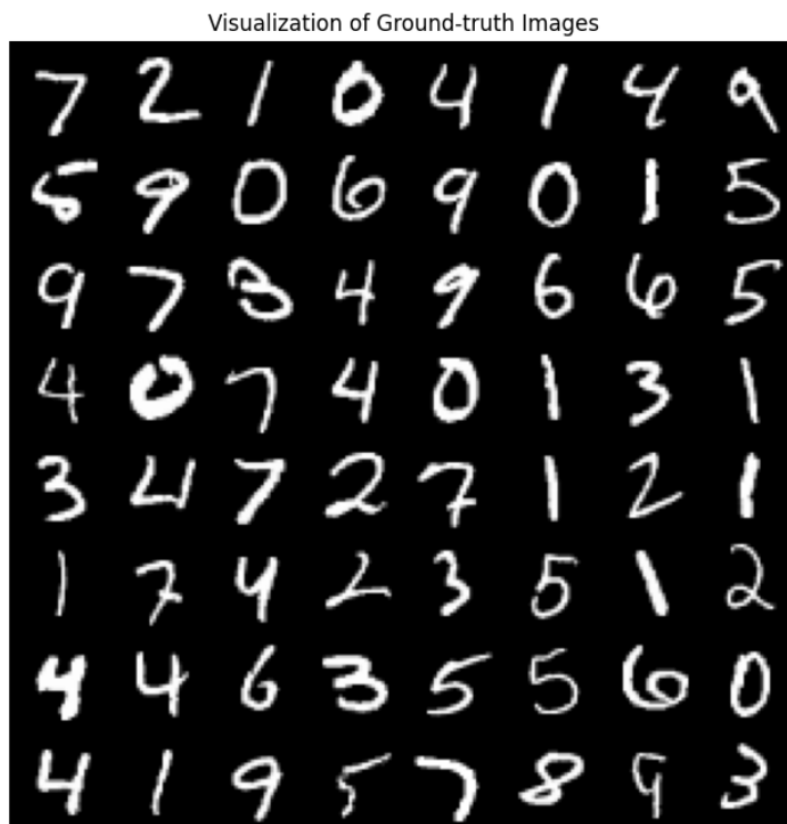
3.1 Tập dữ liệu

Tập dữ liệu MNIST (Modified National Institute of Standards and Technology) là một trong những tập dữ liệu phổ biến nhất trong lĩnh vực học máy, đặc biệt là trong các bài toán phân loại ảnh. MNIST bao gồm các ảnh viết tay của các chữ số từ 0 đến 9, được phân chia thành hai phần: một phần huấn luyện và một phần kiểm tra. Các ảnh trong tập dữ liệu này có kích thước 28x28 pixel và là ảnh đen trắng (grayscale).

Chi tiết:

- Số lượng ảnh: 70,000 ảnh (60,000 ảnh cho tập huấn luyện, 10,000 ảnh cho tập kiểm thử)
- Kích thước ảnh: Mỗi ảnh có kích thước 28x28 pixel, với giá trị pixel nằm trong khoảng từ 0 (đen) đến 255 (trắng), đại diện cho độ sáng của mỗi điểm ảnh trong ảnh.
- Phân loại: Mỗi ảnh trong tập dữ liệu MNIST tương ứng với một chữ số từ 0 đến 9, tức là có tổng cộng 10 lớp (labels) cho mỗi ảnh.

Vì tài nguyên tính toán hạn chế và quá trình huấn luyện cũng như sinh dữ liệu của các mô hình được sử dụng yêu cầu chi phí tính toán lớn, do vậy nhóm quyết định sử dụng tập dữ liệu MNIST có dạng dữ liệu ảnh đơn giản, kích thước ảnh nhỏ để minh họa cho việc huấn luyện và sinh dữ liệu.



Hình 9: Các mẫu trong tập dữ liệu MNIST

3.2 Tổng quan hiện thực và kết quả của các mô hình

Đường dẫn đến thư mục chứa phần hiện thực của nhóm: [DRIVE](#)

3.2.1 Mô hình GAN

3.2.1.a Cấu hình

- `mnist_dim = 28*28`
- `latent_dim = 100`
- `batch_size = 64`
- `learning_rate = 0.0002`
- `epochs = 200`
- `criterion = nn.BCELoss()`

- `d_optimizer = Adam`
- `g_optimizer = Adam`
- `device: GPU Nvidia Tesla T4`

3.2.1.b Kiến trúc

1. Discriminator:

- `fc1: Linear(784, 1024, bias=True)`
- `fc2: Linear(1024, 512, bias=True)`
- `fc3: Linear(512, 256, bias=True)`
- `fc4: Linear(256, 1, bias=True)`

Kiến trúc của Discriminator bao gồm bốn lớp fully connected (FC). Lớp đầu tiên (`fc1`) nhận đầu vào 784 đơn vị và xuất ra 1024 đơn vị, tiếp theo là `fc2` ($1024 \rightarrow 512$), `fc3` ($512 \rightarrow 256$), và cuối cùng `fc4` ($256 \rightarrow 1$), với mỗi lớp có tham số bias. Mô hình này được thiết kế để phân loại đầu vào là thật hay giả trong mạng GAN.

2. Generator:

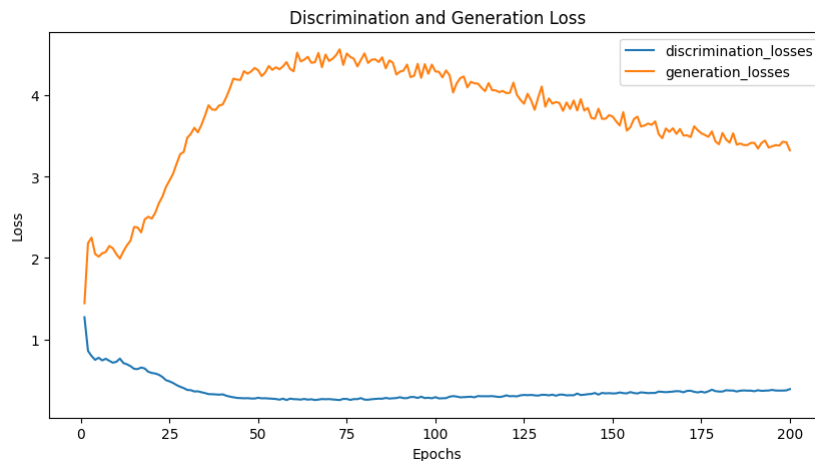
- `fc1: Linear(100, 256, bias=True)`
- `fc2: Linear(256, 512, bias=True)`
- `fc3: Linear(512, 1024, bias=True)`
- `fc4: Linear(1024, 784, bias=True)`

Kiến trúc của Generator bao gồm bốn lớp fully connected (FC). Lớp đầu tiên (`fc1`) nhận đầu vào 100 đơn vị và xuất ra 256 đơn vị, tiếp theo là `fc2` ($256 \rightarrow 512$), `fc3` ($512 \rightarrow 1024$), và cuối cùng `fc4` ($1024 \rightarrow 784$), với mỗi lớp có tham số bias. Mô hình này được thiết kế để tạo ra dữ liệu giả từ một vectơ ngẫu nhiên đầu vào, thường được sử dụng trong mạng GAN để sinh ra hình ảnh.

3.2.1.c Kết quả

- **Thời gian huấn luyện:** tổng thời gian huấn luyện là 47 phút, tương đương 14s/epoch.

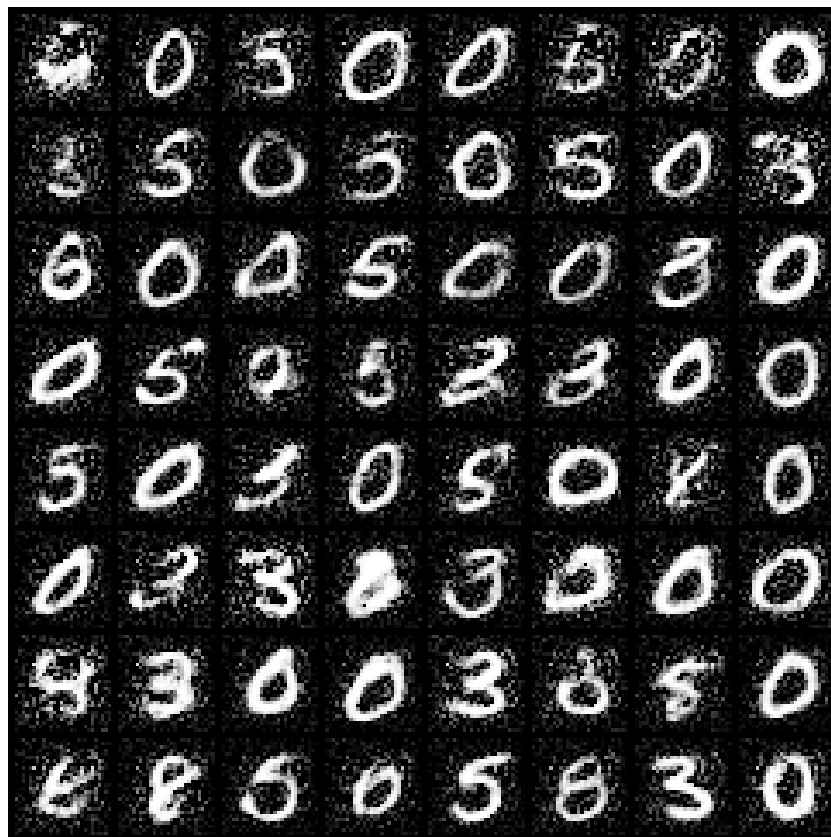
- **Giá trị hàm loss:** Sau 200 epoch, giá trị hàm loss tại discrimination là 0.386, generation 3.326



Hình 10: Giá trị loss trong quá trình huấn luyện GAN

Biểu đồ trên thể hiện sự thay đổi của mất mát phân biệt (discrimination_losses) và mất mát sinh (generation_losses) trong quá trình huấn luyện. Ban đầu, loss của bộ phân biệt giảm nhanh chóng trong khoảng 20 epoch đầu, cho thấy mô hình học hiệu quả trong việc phân biệt dữ liệu thực và giả, sau đó dao động nhẹ và ổn định ở mức thấp, cho thấy sự hội tụ. Ngược lại, loss của bộ sinh tăng mạnh trong giai đoạn đầu (đến khoảng epoch 75) và sau đó dao động ở giảm dần, cho thấy bộ sinh gặp khó khăn trong việc đánh lừa bộ phân biệt. Sự chênh lệch lớn giữa hai mất mát này có thể là dấu hiệu bộ phân biệt quá mạnh so với bộ sinh, dẫn đến sự mất cân bằng trong quá trình huấn luyện và gây khó khăn cho bộ sinh trong việc cải thiện chất lượng dữ liệu.

- **Hình ảnh dữ liệu được sinh ra:**



Hình 11: Hình ảnh dữ liệu được sinh ra bởi GAN

Hình ảnh được sinh ra bởi mô hình chính xác, có thể nhận diện bằng mắt thường, tuy nhiên hình ảnh tương đối nhòe, không đạt được chất lượng cao so với bản gốc.

- **FID Score:** 243.88.

Điều này cho thấy mô hình GAN đạt chất lượng tái tạo ảnh ở mức yếu. Giá trị này cho thấy ảnh được tái tạo vẫn còn khá khác biệt nhiều so với ảnh gốc.

3.2.2 Mô hình Normalizing Flow với RealNVP

3.2.2.a Cấu hình

- $\text{EMBEDDING_DIM} = 20$
- $\text{FLOW_N} = 9$
- $\text{RNVP_TOPOLOGY} = [200]$

- AE_EPOCHS = 10
- NF_EPOCHS = 20
- BATCH_SIZE = 100 Batch size

3.2.2.b Kiến trúc

1. **AutoEncoder:** Tạo embedding của hình ảnh để làm đầu vào cho normalizing flow.
2. **LinearBatchNorm:** Chuẩn hóa batch (batch normalization) đảo ngược, giúp duy trì tính khả nghịch trong normalizing flow.
3. **LinearCouplingLayer:** Sử dụng một mô hình coupling affine để tạo ra biến đổi ngược lại giữa hai phần của dữ liệu đầu vào, giúp tạo sự khả nghịch trong normalizing flow.
4. **Permutation:** Thực hiện phép hoán đổi (permutation) trên các chiều của dữ liệu.
5. **SequentialFlow:** Tạo ra một chuỗi các phép biến đổi liên tiếp trong normalizing flow.
6. **LinearRNVP:** Mô hình RNVP (RealNVP), kết hợp các lớp coupling affine với các bước hoán đổi và chuẩn hóa batch để tạo thành một normalizing flow hoàn chỉnh.

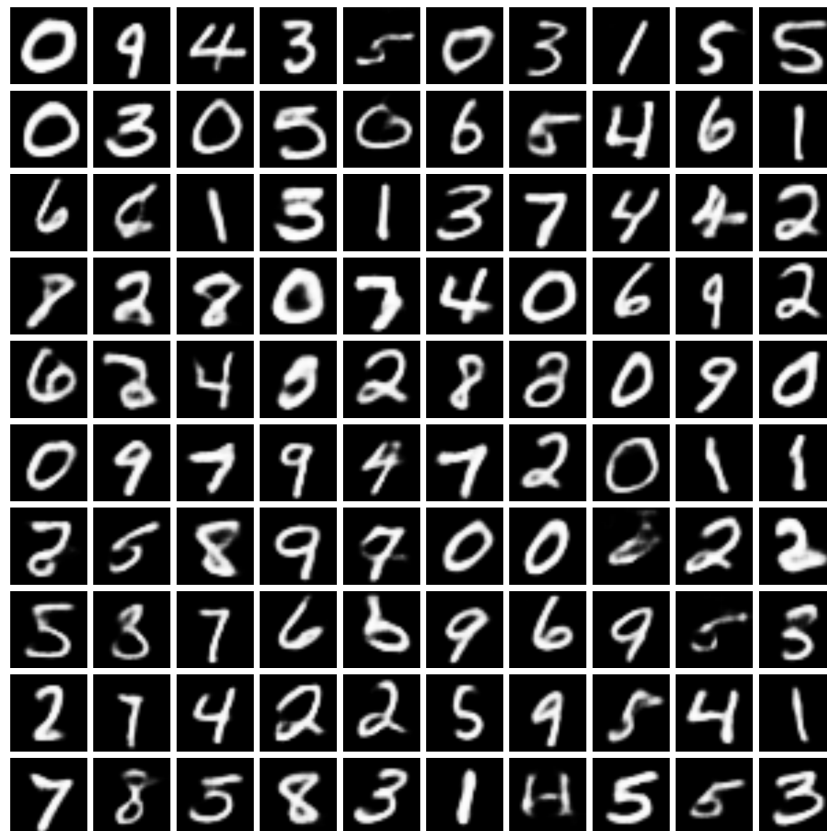
3.2.2.c Kết quả

Quá trình huấn luyện được diễn ra với:

- GPU Nvidia 1650 Ti (4GB VRAM)
- 20 epochs
- Thời gian huấn luyện: ≈ 10 phút

Kết quả mô hình cho ra: FID = 29.92

Ảnh được sinh ra bởi mô hình Normalizing FLOW với RealNVP có chất lượng khá tốt, ảnh tái tạo giống phần lớn với tập huấn luyện.



Hình 12: Kết quả sinh ảnh với mô hình Normalizing Flow

3.2.3 Mô hình VAE

3.2.3.a Cấu hình

- input_dim=784
- batch_size = 64
- latent_dim = 2
- lr = 5e-5
- epochs = 200
- optimizer = Adam
- device: GPU Nvidia Tesla T4

3.2.3.b Kiến trúc

1. Encoder:

- (fc1): `Linear(in_features=784, out_features=512, bias=True)`
- (fc2): `Linear(in_features=512, out_features=256, bias=True)`
- (fc31): `Linear(in_features=256, out_features=2, bias=True)`
- (fc32): `Linear(in_features=256, out_features=2, bias=True)`

Encoder được thiết kế để mã hóa ảnh đầu vào thành một không gian tiềm ẩn (latent space) có kích thước cố định. Thành phần này bao gồm các lớp Fully Connected (FC) được kết hợp với hàm kích hoạt ReLU, giúp biến đổi ảnh từ dạng tensor có kích thước $1 \times 28 \times 28$ thành một vector đặc trưng gọn gàng hơn. Kết quả cuối cùng của encoder là hai vector đại diện cho giá trị trung bình (mean) và phương sai (variance), dùng để định nghĩa một phân phối xác suất trong không gian tiềm ẩn.

2. Decoder:

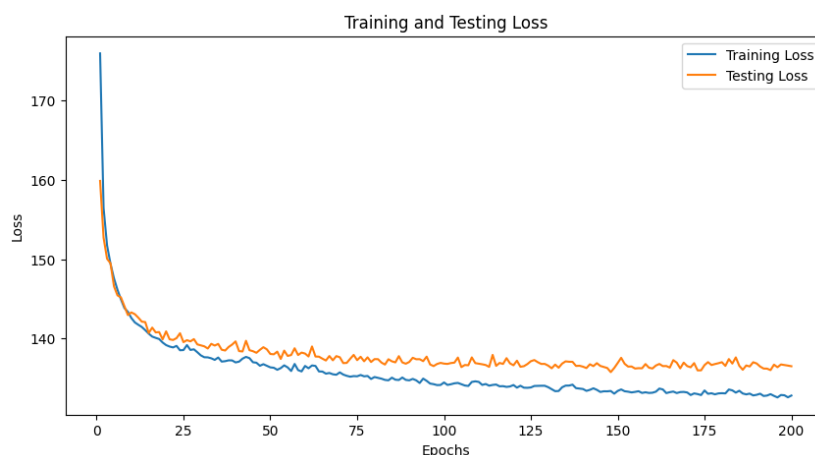
- (fc4): `Linear(in_features=2, out_features=256, bias=True)`
- (fc5): `Linear(in_features=256, out_features=512, bias=True)`
- (fc6): `Linear(in_features=512, out_features=784, bias=True)`

Decoder chịu trách nhiệm tái tạo lại ảnh từ vector trong không gian tiềm ẩn. Bằng cách sử dụng các lớp Fully Connected (FC) và hàm kích hoạt Sigmoid, decoder biến đổi vector latent trở lại kích thước ảnh gốc $1 \times 28 \times 28$. Kết quả là một ảnh tái tạo được dự đoán dựa trên vector latent đã sinh.

3. Variational Autoencoder (VAE): kết hợp cả encoder và decoder để thực hiện hai nhiệm vụ chính: mã hóa và giải mã.

3.2.3.c Kết quả

- **Thời gian huấn luyện:** tổng thời gian huấn luyện là 33 phút, tương đương 11s/epoch.
- **Giá trị hàm loss:** Sau 200 epoch, giá trị hàm loss cho tập rain là 157.0616, tập test là 136.4882



Hình 13: Giá trị loss trong quá trình huấn luyện VAE

Biểu đồ trên minh họa sự thay đổi của Training Loss và Testing Loss qua 200 epochs. Trong giai đoạn đầu (0-25 epochs), cả hai loại loss giảm mạnh, cho thấy mô hình học được các đặc trưng cơ bản từ dữ liệu. Đến giai đoạn giữa (25-100 epochs), Training Loss tiếp tục giảm đều, trong khi Testing Loss dao động nhẹ nhưng không giảm đáng kể, thể hiện mô hình dần ổn định. Ở giai đoạn cuối (100-200 epochs), Training Loss tiếp tục giảm nhưng Testing Loss có xu hướng giữ nguyên hoặc dao động, có khả năng là dấu hiệu của overfitting.

- **Hình ảnh dữ liệu được sinh ra:**



Hình 14: Hình ảnh dữ liệu được sinh ra bởi VAE

Hình ảnh được sinh ra bởi mô hình chính xác, có thể nhận diện bằng mắt thường, tuy nhiên hình ảnh tương đối mờ, không đạt được chất lượng cao so với bản gốc.

- **FID Score:** 138.50.

Điều này cho thấy mô hình VAE đạt chất lượng tái tạo ảnh ở mức trung bình, chưa thực sự tốt. Giá trị này cho thấy ảnh được tái tạo vẫn còn khá khác biệt so với ảnh gốc.

3.2.4 Mô hình Diffusion

3.2.4.a Cấu hình

- timestep_embedding_dim = 256
- n_layers = 8
- hidden_dim = 256

- $n_timesteps = 1000$
- $beta_minmax = [1e-4, 2e-2]$
- $train_batch_size = 128$
- $inference_batch_size = 64$
- $lr = 5e-5$
- $epochs = 200$

3.2.4.b Kiến trúc

1. **SinusoidalPosEmb:** Tạo embedding cho bước thời gian t dưới dạng hàm sin và cosin, cung cấp ngữ cảnh về mức độ nhiễu theo thời gian cho mô hình.
2. **Denoiser:** Dự đoán nhiễu đã được thêm vào ảnh trong quá trình khuếch tán, từ đó giúp tái tạo lại ảnh trong quá trình khuếch tán nghịch.
 - Nhận đầu vào là một ảnh nhiễu và bước thời gian khuếch tán t .
 - Sử dụng các lớp tích chập (ConvBlock) đơn giản hơn để học được nhiễu của ảnh thay vì sử dụng kiến trúc giống U-Net.
 - Tích hợp embedding của bước thời gian t để có thêm thông tin về mức độ nhiễu.
 - Đầu ra là nhiễu được dự đoán.
3. **Diffusion:** Triển khai quá trình khuếch tán thuận và khuếch tán nghịch, bao gồm cả việc sinh mẫu (sampling) và khử nhiễu (denoising).
 - Quá trình khuếch tán thuận: Thêm nhiễu vào ảnh dần dần qua nhiều bước thời gian dựa trên lịch trình phương sai (variance schedule) đã định trước, trả về ảnh bị nhiễu và nhiễu được thêm.
 - Quá trình khuếch tán nghịch: Khử nhiễu ảnh bắt đầu từ nhiễu ngẫu nhiên để tạo ra ảnh sạch (x_0), sử dụng mô hình Denoiser để dự đoán nhiễu tại mỗi bước thời gian.
 - Sinh mẫu: Bắt đầu từ nhiễu ngẫu nhiên và khử nhiễu từng bước để tạo ra ảnh thực tế.

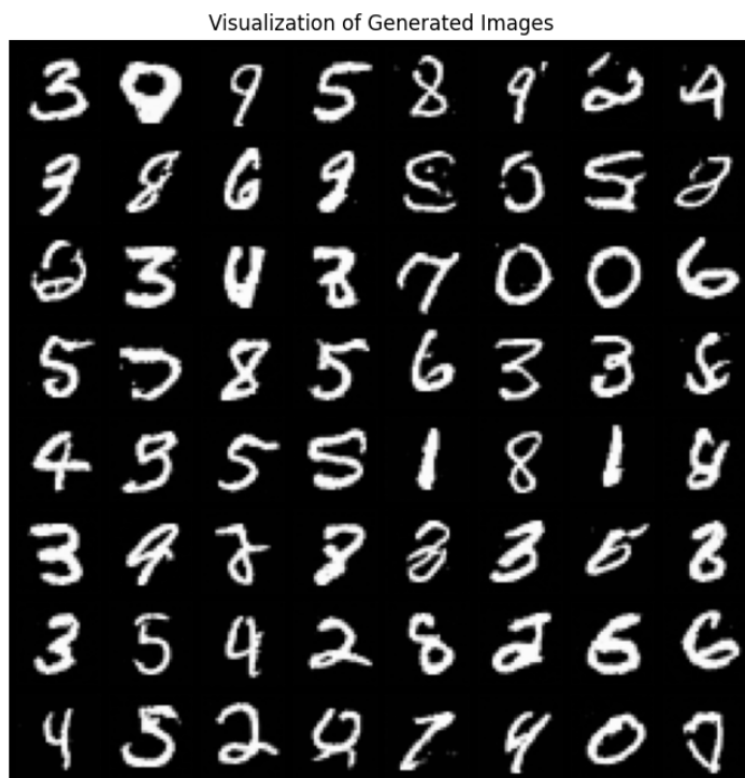
3.2.4.c Kết quả

Quá trình huấn luyện được diễn ra với:

- GPU Nvidia Tesla P100 (16GB VRAM)
- 200 epochs
- Thời gian huấn luyện: ≈ 10 tiếng
- Thời gian sinh mẫu: ≈ 1 ảnh/giây

Kết quả mô hình cho ra: FID = 37.53

Điều này cho thấy mô hình VAE đạt chất lượng tái tạo ảnh ở mức khá tốt. Giá trị FID này cho thấy tập ảnh được sinh ra giống phần lớn so với tập dữ liệu gốc.



Hình 15: Kết quả sinh ảnh với mô hình Diffusion

4 SO SÁNH, ĐÁNH GIÁ KẾT QUẢ GIỮA CÁC MÔ HÌNH

Bảng dưới đây tóm tắt kết quả của các mô hình dựa trên chỉ số Frechet Inception Distance (FID) và thời gian huấn luyện.

Mô hình	FID	Thời gian huấn luyện
GAN	243.88	47 phút (14s/epoch)
Normalizing Flow	29.92	1 tiếng 40 phút (20s/epoch)
VAE	138.50	33 phút (11s/epoch)
Diffusion	37.53	10 tiếng (180s/epoch)

Bảng 1: So sánh các mô hình trong việc sinh ảnh

Phân tích chi tiết

1. GAN (Generative Adversarial Networks)

GAN nổi bật nhờ cơ chế học đối kháng giữa hai mạng: Generator (tạo dữ liệu giả) và Discriminator (phân biệt thật/giả). Dù vậy, trong kết quả thực nghiệm này, GAN có FID cao nhất (243.88), phản ánh chất lượng ảnh sinh ra kém so với dữ liệu thực. FID cao có thể là do sự mất cân bằng giữa Generator và Discriminator đã làm giảm chất lượng ảnh hoặc cũng có thể bị Mode Collapse trong quá trình train. Điều này cho thấy GAN cần các kỹ thuật ổn định hóa (như Gradient Penalty hoặc cải tiến kiến trúc) để đạt được kết quả tốt hơn. Có thể kết luận rằng, GAN phù hợp trong các bài toán cần sự linh hoạt và nhanh chóng, nhưng yêu cầu kỹ thuật tối ưu hóa để khai thác hết tiềm năng.

2. Normalizing Flow với RealNVP

Mô hình này đạt FID tốt nhất (29.92), cho thấy khả năng học phân phối chính xác và sinh ảnh có chất lượng cao. Ưu điểm chính của Normalizing Flow là khả năng học mật độ xác suất trực tiếp, giúp sinh dữ liệu có phân phối sát với dữ liệu thực. Hạn chế của Normalizing Flow là mô hình đòi hỏi phải tính toán định thức

Jacobian trong quá trình huấn luyện, dẫn đến yêu cầu tài nguyên cao. Ngoài ra, hiệu suất có thể giảm nếu áp dụng trên dữ liệu phức tạp hoặc có độ phân giải lớn. Mô hình này lý tưởng trong các bài toán cần sự chính xác và ổn định về phân phối, nhưng không thích hợp khi tài nguyên hạn chế.

3. VAE (Variational Autoencoder)

VAE đạt FID trung bình (138.50) với thời gian huấn luyện ngắn. Đây là mô hình đơn giản, dễ triển khai và phù hợp trong các bài toán không yêu cầu chất lượng ảnh quá cao. Về hạn chế, do giả định về không gian tiềm ẩn (Gaussian) và sự đánh đổi giữa tái tạo và tổng quát hóa, ảnh sinh ra thường thiếu chi tiết và mờ hơn so với GAN hoặc Diffusion Models. VAE phù hợp cho các ứng dụng cần sự đơn giản, nhanh chóng, và tài nguyên thấp, nhưng không lý tưởng nếu chất lượng ảnh là yếu tố quan trọng.

4. Diffusion Models

Diffusion Models đạt FID thấp thứ hai (37.53), cho thấy khả năng sinh ảnh chất lượng cao và chi tiết. Quá trình sinh ảnh của Diffusion Models khởi nhiều từng bước, giúp tái tạo chính xác các đặc trưng phức tạp trong dữ liệu. Mặc dù có chất lượng vượt trội, Diffusion Models yêu cầu thời gian huấn luyện dài (10 tiếng) và tài nguyên lớn, làm tăng chi phí triển khai. Đây là lựa chọn tốt nhất cho các bài toán yêu cầu chất lượng ảnh cao, đặc biệt khi tài nguyên không phải vấn đề lớn.

5 TỔNG KẾT

Nghiên cứu đã tập trung vào việc tổng hợp, hiện thực và đánh giá bốn mô hình học sâu tiêu biểu cho bài toán sinh ảnh, bao gồm Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), Normalizing Flow, và Diffusion Models. Các mô hình này được triển khai và kiểm tra trên tập dữ liệu MNIST để đảm bảo tính khả thi và đánh giá hiệu quả. Kết quả thực nghiệm đã mang lại những phát hiện quan trọng về ưu điểm, hạn chế và khả năng ứng dụng của từng mô hình.

Về hiệu suất, mô hình Normalizing Flow đạt kết quả tốt nhất với chỉ số Frechet Inception Distance (FID) là 29.92. Điều này cho thấy khả năng vượt trội trong việc học phân phối dữ liệu và sinh ảnh chất lượng cao. Mô hình Diffusion đứng thứ hai với FID là 37.53, nhờ khả năng khử nhiễu từng bước, tái tạo hình ảnh chi tiết và chính xác. Tuy nhiên, cả hai mô hình này đều đòi hỏi tài nguyên tính toán lớn, đặc biệt là Diffusion với thời gian huấn luyện kéo dài.

Trong khi đó, VAE đạt FID là 138.50, phản ánh chất lượng sinh ảnh ở mức trung bình. Mô hình này đơn giản, dễ triển khai, nhưng hạn chế ở giả định Gaussian trong không gian tiềm ẩn, dẫn đến ảnh sinh ra thiếu chi tiết. GAN đạt FID cao nhất (243.88), cho thấy chất lượng sinh ảnh kém nhất trong các mô hình được so sánh. Hiện tượng bất ổn định trong huấn luyện và "mode collapse" làm giảm hiệu quả của GAN, dù đây là mô hình linh hoạt và phổ biến.

Kết quả nghiên cứu đã chỉ ra rằng mỗi mô hình có những ưu điểm và hạn chế riêng, phù hợp với từng ứng dụng cụ thể. Normalizing Flow và Diffusion Models là lựa chọn tối ưu khi cần chất lượng ảnh cao và độ chính xác trong phân phối, nhưng đòi hỏi tài nguyên lớn. VAE phù hợp với các bài toán đơn giản, yêu cầu ít tài nguyên, trong khi GAN cần cải tiến về ổn định hóa để phát huy tiềm năng trong các ứng dụng thực tiễn.

Hướng phát triển trong tương lai có thể bao gồm việc cải tiến GAN bằng các kỹ thuật như Gradient Penalty hoặc sử dụng kiến trúc Transformer để nâng cao hiệu quả. Ngoài ra, việc kết hợp ưu điểm của Diffusion Models với các cơ chế giảm chi phí tính toán có thể cải thiện tính khả thi trong thực tế. Cuối cùng, thử nghiệm trên các tập dữ liệu phức tạp hơn sẽ giúp đánh giá khả năng tổng quát hóa của các mô hình, mở rộng ứng dụng trong nhiều lĩnh vực.

Nghiên cứu này đã đóng góp một cái nhìn toàn diện và hệ thống về các mô hình sinh ảnh, cung cấp cơ sở cho các nghiên cứu và ứng dụng tiếp theo trong lĩnh vực trí tuệ nhân tạo.

Tài liệu tham khảo

- [1] Lilian Weng, *What are Diffusion Models?*, <https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>.
- [2] Lilian Weng, *https://lilianweng.github.io/posts/2018-08-12-vae/*, <https://lilianweng.github.io/posts/2018-08-12-vae/>.
- [3] Lilian Weng, *Flow-based Deep Generative Models*, <https://lilianweng.github.io/posts/2018-10-13-flow-models/>.
- [4] Lilian Weng, *From GAN to WGAN*, <https://lilianweng.github.io/posts/2017-08-20-gan/>.
- [5] Jonathan Ho, Ajay Jain, Pieter Abbeel, *Denoising Diffusion Probabilistic Models (DDPM)*. NeurIPS 2020, <https://arxiv.org/abs/2006.11239>.
- [6] Diederik P Kingma, Max Welling, *Auto-Encoding Variational Bayes*, <https://arxiv.org/abs/1312.6114>.